Expected Value Football Betting System

Complete Workflow: StatsBomb × Dixon–Coles × The Odds API

Matthew Kearney

March 2026

Contents

1 System Overview **3**

2 High-Level Pipeline **3**

3 Step 1: Data Collection via StatsBomb Open Data **3**
3.1 Available Endpoints . . . 3
3.2 Fields to Extract . . . 4
3.3 Aggregating xG per Match . . . 4

4 Step 2: Build the Historical Match Table **4**

5 Step 3: Estimate Dixon–Coles Parameters via MLE **4**
5.1 Parameters . . . 4
5.2 Expected Goals Per Match . . . 5
5.3 Time-Weighting . . . 5
5.4 Dixon–Coles Correction Factor . . . 5
5.5 Weighted Log-Likelihood . . . 5
5.6 Identifiability Constraints . . . 5
5.7 Manual Warm-Start Estimates (Before Running MLE) . . . 6
5.8 Optimisation Setup . . . 6

6 Step 4: Compute the Score Probability Matrix **6**
6.1 Joint Score Probability . . . 6
6.2 Building the Matrix . . . 6

7 Step 5: Derive Match Outcome Probabilities **6**

8 Step 6: Pull Live Odds from The Odds API **7**
8.1 Endpoint . . . 7
8.2 Response Fields to Extract . . . 7
8.3 Convert Decimal Odds to Implied Probability . . . 7

9 Step 7: Calculate Expected Value per Bet **8**
9.1 EV Formula . . . 8
9.2 Worked Example . . . 8
9.3 Bet Screening Criteria . . . 8

10 Step 8: Kelly Criterion Stake Sizing **8**
10.1 Full Kelly Formula . . . 8
10.2 Fractional Kelly . . . 9
10.3 Worked Example (Continued from Step 7) . . . 9
10.4 Hard Caps . . . 9

11 Step 9: Output and Dashboard **9**
11.1 Data to Present Per Value Bet . . . 9
11.2 Example Output Table . . . 9
11.3 Model Refresh Schedule . . . 10

12 Complete Formula Reference **10**

13 Technology Stack Summary **10**

1 System Overview

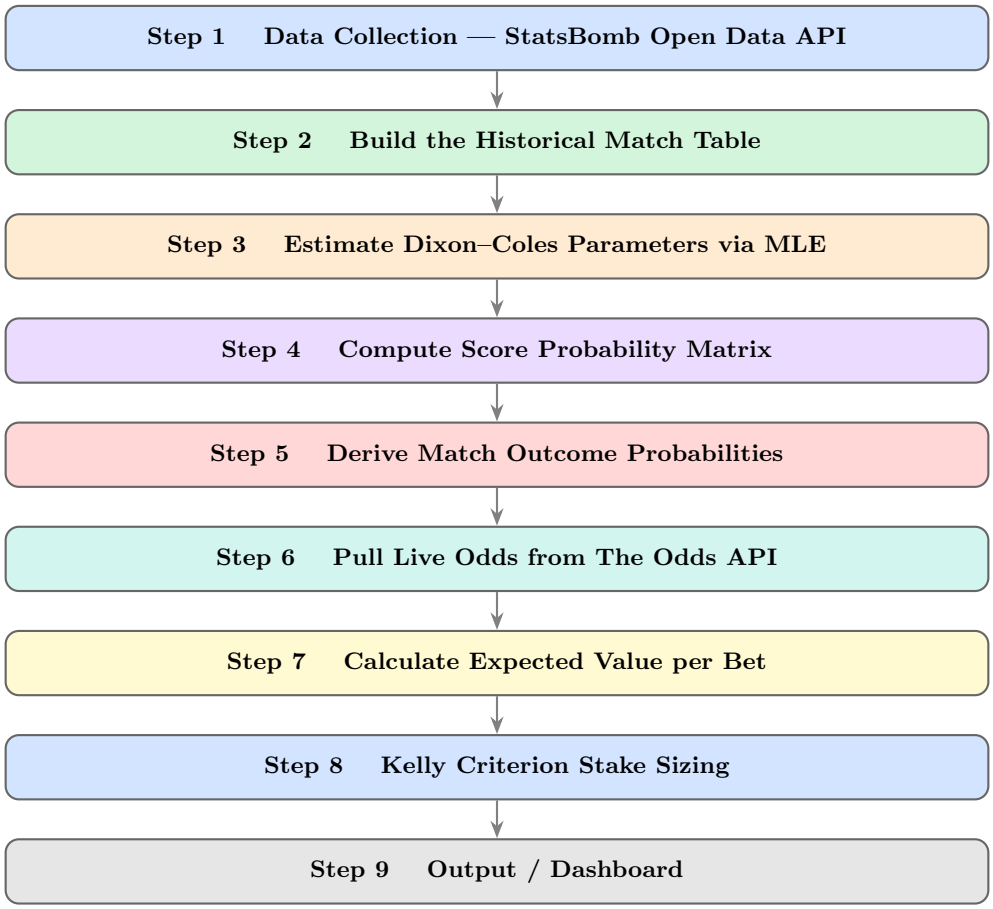
This document is the complete, step-by-step engineering and mathematical reference for building a football expected-value (EV) betting system.

Three data sources drive the system:

- **StatsBomb Open Data API** — historical match event data, including per-shot xG values.
- **Dixon–Coles Model** — statistical model that converts team strengths into true score probabilities.
- **The Odds API** — real-time sportsbook odds across multiple bookmakers.

Why xG instead of actual goals?
Actual scorelines contain significant noise (lucky deflections, keeper errors). Expected Goals (xG) measures shot quality and better reflects a team's true attacking and defensive performance, producing more stable parameter estimates.

2 High-Level Pipeline



3 Step 1: Data Collection via StatsBomb Open Data

3.1 Available Endpoints

Endpoint	Returns
`GET /api/v3/competitions`	All available leagues and seasons
`GET /api/v3/matches?competition_id={id}&season_id={id}`	All matches in a season
`GET /api/v3/events?match_id={id}`	All events in a match (shots, passes, etc.)

3.2 Fields to Extract

From the **matches** endpoint, record for every game:

- `match_date` — used for time-weighting decay
- `home_team.home_team_name`
- `away_team.away_team_name`
- `home_score`, `away_score`

From the **events** endpoint, filter on `type.name == "Shot"` and extract:

- `team.name` — which team took the shot
- `shot.statsbomb_xg` — xG value for that shot

3.3 Aggregating xG per Match

Sum shot-level xG into two team totals per game:

$$\mathrm{xG}_{\text{home}} = \sum_{\substack{k \in \text{shots} \\ \text{team=home}}} \texttt{shot.statsbomb_xg}_k \qquad \mathrm{xG}_{\text{away}} = \sum_{\substack{k \in \text{shots} \\ \text{team=away}}} \texttt{shot.statsbomb_xg}_k$$

4 Step 2: Build the Historical Match Table

Each row in the training dataset represents one match:

Date	Home	Away	Home xG	Away xG	Days Ago
2024-09-12	Arsenal	Chelsea	1.83	0.91	192
2024-10-05	Man City	Liverpool	2.31	1.44	169
⋮	⋮	⋮	⋮	⋮	⋮

Recommended dataset size: at least the most recent full season (38 matchdays × 10 games = 380 matches for the Premier League). Using two seasons improves stability for teams with fewer matches.

5 Step 3: Estimate Dixon–Coles Parameters via MLE

5.1 Parameters

For a league with N teams, the model has $2N + 2$ free parameters:

Symbol	Meaning	One value per
α_i	Attack strength	Team i
β_i	Defensive strength	Team i
γ	Home advantage multiplier	Whole league
ρ	Low-score correlation	Whole league

5.2 Expected Goals Per Match

$$\lambda = \alpha_{\text{home}}\,\beta_{\text{away}}\,\gamma \qquad \text{(expected xG for home team)}$$
$$\mu = \alpha_{\text{away}}\,\beta_{\text{home}} \qquad \text{(expected xG for away team)}$$

A high α means a strong attack. A low β means a strong defence (concedes fewer goals relative to average).

5.3 Time-Weighting

Give more weight to recent matches using exponential decay:

$$w_k = e^{-\xi \cdot d_k}$$

where d_k is the number of days since match k and $\xi = 0.0065$ is the standard decay constant. At this rate:

- A match played 90 days ago has weight ≈ 0.56
- A match played 180 days ago has weight ≈ 0.31
- A match played 365 days ago has weight ≈ 0.09

5.4 Dixon–Coles Correction Factor

The standard independent Poisson model systematically underestimates 0–0 and 1–1 results and overestimates 1–0 and 0–1 results. The correction factor τ fixes this:

$$\tau_{\lambda,\mu}(x,y) = \begin{cases} 1 - \lambda\mu\rho & x = 0,\ y = 0 \\ 1 + \lambda\rho & x = 0,\ y = 1 \\ 1 + \mu\rho & x = 1,\ y = 0 \\ 1 - \rho & x = 1,\ y = 1 \\ 1 & \text{otherwise} \end{cases}$$

When $\rho = 0$ the model reduces to the standard independent Poisson model. Empirically ρ is typically small and negative ($\rho \approx -0.10$).

5.5 Weighted Log-Likelihood

Maximise this objective function over all M historical matches:

$$\ell(\boldsymbol{\theta}) = \sum_{k=1}^{M} w_k \Big[\log \tau(\lambda_k, \mu_k, x_k, y_k) + x_k \log \lambda_k - \lambda_k - \log(x_k!) + y_k \log \mu_k - \mu_k - \log(y_k!)\Big]$$

where x_k and y_k are the observed xG totals for match k, and $\boldsymbol{\theta} = \{\alpha_i, \beta_i, \gamma, \rho\}$.

5.6 Identifiability Constraints

Without constraints the parameters are not uniquely identified (you can multiply all α_i by any constant and divide all β_i by the same constant and get the same λ, μ). Enforce:

$$\frac{1}{N}\sum_{i=1}^{N} \alpha_i = 1 \qquad \frac{1}{N}\sum_{i=1}^{N} \beta_i = 1$$

In practice, **fix one team's α to 1.0** as the anchor team so the optimiser has a unique solution.

5.7 Manual Warm-Start Estimates (Before Running MLE)

These closed-form approximations are used to initialise parameters. Let $\bar{g}_H$ = league average home xG and $\bar{g}_A$ = league average away xG.

$$\alpha_i = \frac{\text{team } i \text{ avg xG scored (home or away)}}{\text{league avg xG scored (corresponding context)}} \qquad \beta_i = \frac{\text{team } i \text{ avg xG conceded}}{\text{league avg xG conceded (opposing context)}}$$

$$\gamma = \frac{\bar{g}_H}{\bar{g}_A}$$

These are approximations only. MLE refines them to the true optimum.

5.8 Optimisation Setup

1. **Initialise:** $\alpha_i = 1$, $\beta_i = 1$, $\gamma = 1.2$, $\rho = -0.1$ for all teams.
2. **Maximise** $\ell(\boldsymbol{\theta})$ using `scipy.optimize.minimize(method='L-BFGS-B')`, minimising the negative log-likelihood $-\ell$.
3. **Normalise** the converged solution so the mean $\alpha_i = 1$ and mean $\beta_i = 1$.
4. **Validate** by re-running from multiple random starting points and confirming the optimum is stable.

6 Step 4: Compute the Score Probability Matrix

6.1 Joint Score Probability

With λ and μ computed for an upcoming match, the probability of every exact scoreline (x, y) is:

$$P(X = x,\, Y = y) = \tau_{\lambda,\mu}(x, y) \cdot \frac{e^{-\lambda}\lambda^x}{x!} \cdot \frac{e^{-\mu}\mu^y}{y!}$$

6.2 Building the Matrix

Evaluate for $x, y \in \{0, 1, 2, \ldots, 8\}$, producing a 9×9 grid. This range covers more than 99% of real-world match outcomes. Every cell contains the probability of that exact scoreline. The entire matrix must sum to 1.

	$y = 0$	$y = 1$	$y = 2$	$y = 3$	$\cdots$
$x = 0$	$P(0,0)$	$P(0,1)$	$P(0,2)$	$P(0,3)$	$\cdots$
$x = 1$	$P(1,0)$	$P(1,1)$	$P(1,2)$	$P(1,3)$	$\cdots$
$x = 2$	$P(2,0)$	$P(2,1)$	$P(2,2)$	$P(2,3)$	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

The Dixon–Coles correction only modifies the four cells $(0, 0)$, $(0, 1)$, $(1, 0)$, $(1, 1)$. All other cells use the raw Poisson product.

7 Step 5: Derive Match Outcome Probabilities

Aggregate the score matrix into the three canonical bet markets by summing the relevant cells:

$$P(\text{Home Win}) = \sum_{x>y} P(x,y)$$

$$P(\text{Draw}) = \sum_{x=y} P(x,y)$$

$$P(\text{Away Win}) = \sum_{x<y} P(x,y)$$

These three probabilities sum to 1 by construction.

You can extend the same score matrix to other markets:

- **Both Teams to Score (BTTS):** $P(\text{BTTS}) = \sum_{x\geq 1,\, y\geq 1} P(x,y)$
- **Over 2.5 Goals:** $P(\text{Over } 2.5) = \sum_{x+y\geq 3} P(x,y)$
- **Correct Score:** Read directly from cell $P(x,y)$ in the matrix.

8 Step 6: Pull Live Odds from The Odds API

8.1 Endpoint

```
GET https://api.the-odds-api.com/v4/sports/soccer_epl/odds
    ?apiKey={YOUR_KEY}
    &regions=uk,eu,us
    &markets=h2h
    &oddsFormat=decimal
```

Use `markets=h2h` for match result (Home/Draw/Away). Use `markets=totals` for Over/Under markets. Use `markets=btts` for Both Teams to Score.

8.2 Response Fields to Extract

For each bookmaker line, extract:

- `bookmakers[].markets[].outcomes[].name` — "Home", "Draw", or "Away"
- `bookmakers[].markets[].outcomes[].price` — decimal odds (e.g. 2.10)

Always take the **best available price** across all bookmakers for each outcome (line shopping).

8.3 Convert Decimal Odds to Implied Probability

$$P_{\text{market}} = \frac{1}{\text{decimal odds}}$$

The raw implied probabilities sum to more than 1 because bookmakers embed a margin (the "overround"). To see the fair implied probability, normalise:

$$\text{overround} = P_H + P_D + P_A \qquad P_{\text{fair}} = \frac{P_{\text{market}}}{\text{overround}}$$

Important: Use P_{market} (not P_{fair}) when computing EV, because that is the actual price the bookmaker is offering.

9 Step 7: Calculate Expected Value per Bet

9.1 EV Formula

For a \$1 stake on outcome O at decimal odds d:

$$\text{EV} = \big(P_{\text{model}}(O) \times (d-1)\big) - \big((1 - P_{\text{model}}(O)) \times 1\big)$$

Simplified:

$$\text{EV} = P_{\text{model}}(O) \times d \; - \; 1$$

A bet has **positive expected value** when EV > 0, which occurs if and only if:

$$P_{\text{model}}(O) > \frac{1}{d} = P_{\text{market}}$$

9.2 Worked Example

Arsenal vs. Chelsea. The model estimates $P_{\text{model}}(\text{Arsenal win}) = 0.49$. Best available decimal odds for an Arsenal win: 2.10.

Market implied probability:

$$P_{\text{market}} = \frac{1}{2.10} \approx 0.476$$

Expected value per \$1 staked:

$$\text{EV} = (0.49 \times 2.10) - (0.51 \times 1) = 1.029 - 0.510 = +0.519$$

Since EV > 0 and $P_{\text{model}} > P_{\text{market}}$, this is a value bet. Over many such bets, the expected profit is \$0.519 per \$1 staked — assuming the model probability is accurate.

9.3 Bet Screening Criteria

Apply these filters before flagging a bet as actionable:

Filter	**Recommended Threshold**
Expected value	EV > 0
Minimum edge	$P_{\text{model}} - P_{\text{market}} > 0.03$
Minimum model prob.	$P_{\text{model}} > 0.10$
Maximum decimal odds	$d < 10.0$
Minimum bookmakers	At least 3 bookmakers sampled

10 Step 8: Kelly Criterion Stake Sizing

10.1 Full Kelly Formula

The Kelly Criterion chooses the stake fraction that maximises the expected logarithm of wealth, giving optimal long-run bankroll growth:

$$f^* = \frac{bp - q}{b}$$

where:

- $b = d - 1$ (net profit per unit staked at decimal odds d)

- $p = P_{\text{model}}(O)$ (model win probability)
- $q = 1 - p$ (model loss probability)

f^* is the fraction of the current bankroll to stake. If $f^* \leq 0$, do not bet.

10.2 Fractional Kelly

Full Kelly is mathematically optimal but leads to aggressive bet sizing and large drawdowns in practice. Use **quarter Kelly** or **half Kelly**:

$$\text{Stake} = \frac{f^*}{4} \times \text{Bankroll} \qquad \text{(quarter Kelly, recommended)}$$

10.3 Worked Example (Continued from Step 7)

$d = 2.10$, $p = 0.49$, $q = 0.51$.

$$b = 2.10 - 1 = 1.10$$
$$f^* = \frac{(1.10)(0.49) - 0.51}{1.10} = \frac{0.539 - 0.510}{1.10} = \frac{0.029}{1.10} \approx 0.026$$

Full Kelly recommends staking 2.6% of the current bankroll. Quarter Kelly recommends 0.65%.

10.4 Hard Caps

Always impose a hard cap regardless of Kelly output:

- Maximum single bet: 2% of bankroll
- Never bet if $f^* \leq 0$
- Never bet on a market where fewer than 3 bookmakers are available

11 Step 9: Output and Dashboard

11.1 Data to Present Per Value Bet

Field	**Description**
Match	Home vs. Away, kick-off time
Bet market	Home Win / Draw / Away Win / BTTS / Over 2.5
Model probability	$P_{\text{model}}(O)$
Market implied prob.	$P_{\text{market}} = 1/d$
Edge	$P_{\text{model}} - P_{\text{market}}$
Best decimal odds	Highest d across all bookmakers
Best bookmaker	Name of bookmaker offering the best line
EV (per \$1 stake)	$P_{\text{model}} \times d - 1$
Kelly fraction f^*	Recommended full-Kelly fraction
Recommended stake	Quarter Kelly $\times$ Bankroll

11.2 Example Output Table

Match	**Bet**	**Odds**	P_{model}	P_{mkt}	**Edge**	**EV**
Arsenal vs Chelsea	Home Win	2.10	0.490	0.476	+1.4%	+0.029
Liverpool vs Man City	Draw	3.50	0.310	0.286	+2.4%	+0.085
Barcelona vs Atletico	Away Win	4.20	0.275	0.238	+3.7%	+0.155

11.3 Model Refresh Schedule

- **Re-fit parameters** after every completed matchday so that the most recent results are reflected in α_i, β_i, γ, and ρ.
- **Re-fetch odds** as close to kick-off as possible. Lines move most significantly in the final 2–6 hours before a match.
- **Do not bet** if parameter data is more than 7 days stale.

12 Complete Formula Reference

Quantity	**Formula**
Home expected xG	$\lambda = \alpha_{\text{home}}\, \beta_{\text{away}}\, \gamma$
Away expected xG	$\mu = \alpha_{\text{away}}\, \beta_{\text{home}}$
Time weight	$w_k = e^{-\xi d_k}, \quad \xi = 0.0065$
Dixon–Coles correction	$\tau(\lambda, \mu, x, y)$ as defined in Section 3.4
Joint score probability	$P(x, y) = \tau(\lambda, \mu, x, y)\, \dfrac{e^{-\lambda}\lambda^x}{x!}\, \dfrac{e^{-\mu}\mu^y}{y!}$
Home win probability	$P_H = \sum_{x>y} P(x, y)$
Draw probability	$P_D = \sum_{x=y} P(x, y)$
Away win probability	$P_A = \sum_{x<y} P(x, y)$
Over 2.5 probability	$P_{O2.5} = \sum_{x+y\geq 3} P(x, y)$
BTTS probability	$P_{\text{BTTS}} = \sum_{x\geq 1,\, y\geq 1} P(x, y)$
Market implied probability	$P_{\text{mkt}} = 1/d$
Expected value (per \$1)	$\text{EV} = P_{\text{model}} \times d - 1$
Kelly fraction	$f^* = (bp - q)/b, \quad b = d - 1$
Recommended stake	$f^*/4 \times \text{Bankroll}$

13 Technology Stack Summary

Layer	**Tool**	**Purpose**
Data ingestion	Python `requests`	Call StatsBomb & Odds APIs
Data storage	PostgreSQL or SQLite	Store match history & parameters
MLE optimisation	`scipy.optimize.minimize`	Fit Dixon–Coles parameters
Probability calc	`numpy`, `scipy.stats.poisson`	Build 9×9 score matrix
Scheduling	`cron` or Celery	Nightly re-fits, pre-match odds pull
Backend API	FastAPI (Python)	Serve bet recommendations
Dashboard	React + Recharts	Display value bets to user